

ML/DL Ops Assignment 3: Short Report

End-to-End Hugging Face Model Training & Docker Deployment

Akshat Jain
M25CSA003

February 24, 2026

1 Model Selection

Pre-trained Model Used: DistilBERT (`distilbert-base-uncased`) trained on the IMDB dataset.

Reason for Selection: DistilBERT was selected because it provides an excellent balance between predictive performance and computational efficiency. It retains over 95% of BERT's performances while being 40% smaller and 60% faster. This makes it highly suitable for text classification tasks and lightweight enough for seamless containerization and deployment within a Docker environment without excessively bloating the image size.

2 Training Summary

The model was fine-tuned using the Hugging Face **Trainer** API. The complete end-to-end pipeline involved the following steps:

- **Data Preparation:** The Jupyter Notebook was converted into modular Python scripts. The IMDB dataset was loaded, tokenized, and prepared for training.
- **Configuration:** `TrainingArguments` were set up to define hyperparameters such as learning rate, batch size, and the number of epochs.
- **Execution & Logging:** The training loop was executed successfully. Training and evaluation metrics were actively logged.
- **Publishing:** Upon completion, the fine-tuned model weights, tokenizer, and training configurations were automatically pushed to the Hugging Face Hub repository (<https://huggingface.co/Despo08/distilbert-imdb-assignment>).

3 Evaluation Comparison

To ensure the integrity of the deployed model, evaluation was performed in two separate environments.

3.1 Local Evaluation

Evaluated directly after training on the local machine (`eval_results.json`):

- **Accuracy:** 87.18% (0.8718)
- **F1 Score:** 0.8716
- **Loss:** 0.3131
- **Runtime:** 192.18 seconds

3.2 Hugging Face Hub Evaluation via Docker

Evaluated by pulling the published model from the Hugging Face Hub inside an isolated Docker container (`hf_eval_results.json`):

- **Accuracy:** 87.18% (0.8718)
- **F1 Score:** 0.8716
- **Loss:** 0.3131
- **Runtime:** 151.17 seconds

Conclusion: The critical metrics (Accuracy, F1 Score, and Loss) from both the local environment and the Dockerized environment match perfectly. This successful replication verifies that the model was correctly published to the Hugging Face Hub and can be seamlessly pulled and executed in a production-ready containerized pipeline.

4 Challenges Faced

During the completion of this assignment, the following key challenges were encountered and resolved:

- **Docker Image Optimization:** Installing massive dependencies like PyTorch and the Hugging Face ecosystem typically results in very large image sizes. Carefully structuring the `Dockerfile` (using multi-stage builds or lightweight base images) was crucial to keep the container optimized.
- **Environment Consistency:** Converting code from a notebook environment to production-ready Python scripts required resolving path issues and ensuring that environment variables and tokenized data behaved exactly the same way inside the Docker container as they did locally.